Solution: Maximum Sum of 3 Non-Overlapping Subarrays

Problem Statement

Given an array of integers `nums` and an integer `k`, find the starting indices of three non-overlapping subarrays having `maximum sum` and each of a length `k`.

Return the array of size 3 containing the starting indices of such subarrays. If `multiple answers` exist, return the lexicographically smallest answer.

Examples

Example 1:

- **Input**: `nums = [1, 2, 3, 1, 1, 2, 1, 3, 2], k = 2`
- **Expected Output**: `[1, 4, 7]`
- **Justification**: The subarrays would be `[2, 3]`, `[1, 2]`, and `[3, 2]` with sums 5, 3, and 5 respectively.

Example 2:

- **Input**: `nums = [4, 5, 10, 6, 11, 17, 4, 5, 6, 7], k = 3`
- **Expected Output**: `[0, 3, 7]`
- **Justification**: The subarrays would be `[4, 5, 10]`, `[6, 11, 17]`, and `[5, 6, 7]` with sums 19, 34, and 18 respectively.

Example 3:

- **Input**: `nums = [1, 1, 1, 1, 1, 1, 1, 1, 1], k = 2`
- **Expected Output**: `[0, 2, 4]`
- **Justification**: The subarrays would be `[1, 1]`, `[1, 1]`, and `[1, 1]` with each having a sum of 2.

Solution

To solve this problem, we'll utilize dynamic programming. The approach involves breaking down the problem into simpler subproblems, solving each subproblem once, and storing their solutions. We focus on finding the maximum sum for each possible ending position of the first, second, and third subarray. By storing the best possible sums and their corresponding starting indices for the first and second subarrays, we can efficiently calculate the total maximum sum for the third subarray.

This approach is effective because it avoids redundant calculations. Instead of recomputing sums for overlapping sub-problems, we simply refer to previously computed values, greatly reducing the time complexity. Additionally, by keeping track of the indices, we can backtrack to find the exact positions of the subarrays after determining the maximum sum.

Step-by-step Algorithm

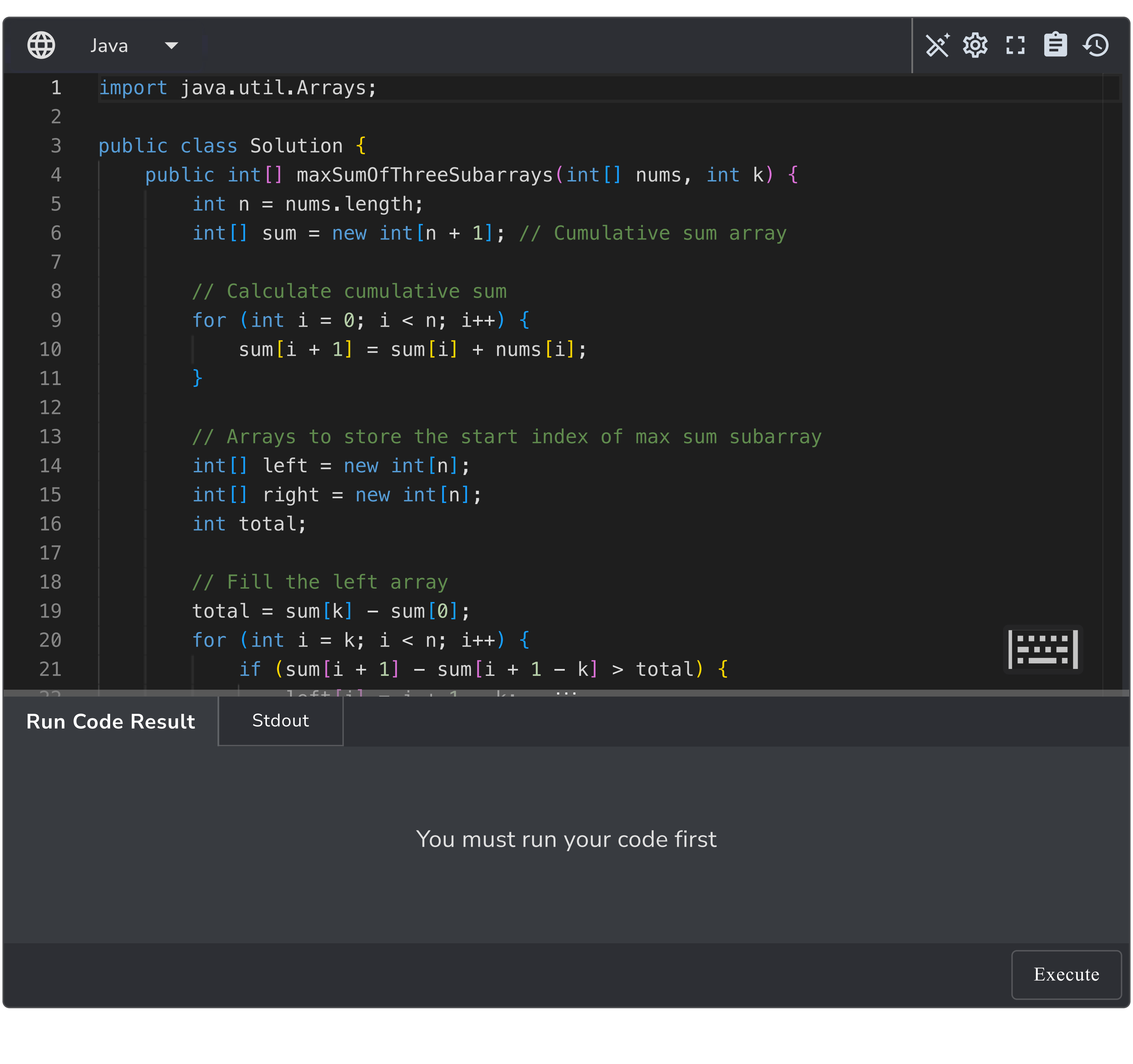
1. **Initialize Cumulative Sum Array** (`sum`):
    - Create an array `sum` to store the cumulative sum of the `nums` array. This helps in calculating the sum of any subarray quickly.
2. **Calculate Cumulative Sum**:
    - Iterate over the `nums` array and fill the `sum` array such that `sum[i]` represents the total sum of the elements from `nums[0]` to `nums[i-1]`.
3. **Initialize `left` Array**:
    - Create an array `left` of the same length as `nums`. The `left[i]` will store the starting index of the subarray that ends at `i` and has the maximum sum among all such subarrays.
4. **Fill the `left` Array**:
    - Iterate through the `nums` array starting from index `k`. For each index `i`, calculate the sum of the subarray ending at `i` (from `i-k+1` to `i`) and update `left[i]` if this subarray has a larger sum than the previous maximum.
5. **Initialize `right` Array**:
    - Similar to `left`, create a `right` array. The `right[i]` will store the starting index of the subarray that begins at `i` and has the maximum sum among all such subarrays.
6. **Fill the `right` Array**:
    - Iterate through the `nums` array in reverse, starting from `n - k`. For each index `i`, calculate the sum of the subarray starting at `i` and update `right[i]` if this subarray has a larger sum than the previous maximum.
7. **Find Maximum Sum of Three Subarrays**:
    - Iterate over the range `[k, n - 2 * k]` and for each index `i`, consider `i` as the ending index of the second subarray. The first subarray must end before `i` and the third must start after `i`. Use `left` and `right` arrays to find the starting indices of these subarrays.
    - Calculate the total sum of these three subarrays. If this total is greater than the previous maximum, update the maximum and store the starting indices of the three subarrays.
8. **Return Result**:
    - After iterating through the entire range, return the stored indices of the three subarrays that give the maximum sum.

Algorithm Walkthrough for Example 1

Using the example `nums = [1, 2, 3, 1, 1, 2, 1, 3, 2]` and `k = 2`:

1. **Initialize and Calculate Cumulative Sum Array** (`sum`):
    - `sum = [0, 1, 3, 6, 7, 8, 10, 11, 14, 16]`
    - Explanation: Each element in `sum` represents the cumulative sum up to that index in `nums`.
2. **Fill the `left` Array**:
    - Start from index `k = 2`. The subarray `[1, 2]` (indices 0 to 1) has a sum of 3.
    - Move to index 3. The subarray `[2, 3]` (indices 1 to 2) has a sum of 5, which is greater than 3. Update `left[3]` to 1.
    - Continue this process. After completion, `left` might look like `[0, 0, 1, 1, 1, 1, 1, 1, 1]`.
3. **Fill the `right` Array**:
    - Start from the end, index `n - k = 7`. The subarray `[3, 2]` (indices 7 to 8) has a sum of 5.
    - Move to index 6. The subarray `[1, 3]` (indices 6 to 7) has a sum of 4, which is less than 5. Keep `right[6]` as 7.
    - Continue in reverse. After completion, `right` might look like `[1, 1, 7, 7, 7, 7, 7, 7, 0]`.
4. **Find Maximum Sum of Three Subarrays**:
    - Iterate from `i = 2` to `i = 5` (as `n - 2 * k = 5`).
        - For `i = 2`: Subarrays are [1, 2] (from `left[1] = 0`), [3, 1] (from `nums[2]` to `nums[3]`), and [3, 2] (from `right[4] = 7`). Total sum is 12.
        - For `i = 3`: Subarrays are [2, 3], [1, 1] (from `nums[3]` to `nums[4]`), and [3, 2] (from `right[5]`). Total sum is 12.
        - For `i = 4`: Subarrays are [2, 3], [1, 2], and [3, 2]. Total sum is 13. This is the maximum sum found so far.
        - For `i = 5`: Subarrays are [1, 2], [2, 1] (from `nums[5]` to `nums[6]`), and [3, 2] (from `right[7]`). Total sum is 11.
5. **Result**:
    - The maximum sum of 13 is achieved with subarrays starting at indices `[1, 4, 7]`.

Code



```java
import java.util.Arrays;

public class Solution {
    public int[] maxSumOfThreeSubarrays(int[] nums, int k) {
        int n = nums.length;
        int[] sum = new int[n + 1]; // Cumulative sum array

        // Calculate cumulative sum
        for (int i = 0; i < n; i++) {
            sum[i + 1] = sum[i] + nums[i];
        }

        // Arrays to store the start index of max sum subarray
        int[] left = new int[n];
        int[] right = new int[n];
        int total;

        // Fill the left array
        total = sum[k] - sum[0];
        for (int i = k; i < n; i++) {
            if (sum[i + 1] - sum[i + 1 - k] > total) {
```

Complexity Analysis

Time Complexity

- **O(n)**: The algorithm iterates over the array multiple times, where `n` is the length of the input array. Despite multiple passes (for cumulative sum, left, right arrays, and finding the maximum sum), each pass is linear, leading to a total time complexity of `O(n)`.

Space Complexity

- **O(n)**: Additional space is used for the cumulative sum array, left array, and right array, each of size `n`. Thus, the total space complexity is `O(n)`, primarily due to these auxiliary data structures.

← Previous
Maximum Sum Of 3 Non-Overlapping Subarrays (Hard)
Next →
Making A Large Island (Hard)
☐ Mark as Completed